

FULL PROJECT REPORT

Name: Tavishi Garg

Title: Campus Emergency Response
& Resource Allocation System (CERA)

Reg No: 25BAI10085

Branch: B. Tech CSE (AI & ML)

Year: 2st Year

Course Code: CSE2006

Course Name: Programming in Java

Date: 18th September 2026

1. Introduction

Campus emergencies such as medical incidents, fire hazards, security threats, electrical problems and other unexpected situations require a very quick response. In a place like college such emergency information may need to be recorded, prioritized, assigned to an appropriate responder and also supported with available resources.

The **Campus Emergency Response & Resource Allocation System (CERA)** is a Java based software system designed to organize the management of emergencies within the campus. The system provides a structured process for recording emergencies, classifying them according to their type and severity, calculating their priority, assigning suitable responders, allocating emergency resources, tracking their status, and generating emergency reports.

CERA uses object-oriented programming concepts to represent different types of users, responders, emergencies, and resources. It also demonstrates Java concepts such as **inheritance, abstraction, polymorphism, interfaces, enumerations, collections, exception handling, file handling, packages and multithreading**.

The system operates as a Java console based application. Emergency reports are maintained using file-based storage while the collections are used during program execution for managing users, emergencies, responders and resources.

2. Problem Statement

In a college campus, emergencies such as medical incidents, fire hazards and security threats require immediate and timely coordination between people and available resources. Handling these incidents manually or through disconnected communication can make it difficult to identify the emergency type, determine its priority, assign an appropriate responder, check resource availability and track the progress of an emergency.

A structured software system is therefore required to organize these activities into a logical workflow.

CERA addresses this problem by providing a centralized Java based system in which campus emergencies can be recorded and processed systematically. The system classifies emergencies, calculates their priority, assigns suitable responders, manages emergency resources, updates emergency status, handles exceptional situations and stores emergency reports.

3. Functional Requirements

The CERA system contains multiple functional modules that work together to manage campus emergencies.

3.1 User and Responder Management

The system should allow different types of campus users and responders to be represented and managed.

Inputs:

- User ID
- Name
- Email
- User type
- Course/Department
- Responder details

Output:

- User information
 - User type
 - Responder availability
-

3.2 Emergency Reporting

The system should allow an emergency to be recorded with relevant information.

Inputs:

- Emergency ID
- Emergency type
- Location
- Severity
- Description
- Emergency specific information

Output:

- Emergency record containing all provided information.
-

3.3 Emergency Classification

The system should classify emergencies into predefined categories.

The supported emergency types are:

- Medical
- Fire
- Security
- Electrical
- Natural Disaster

- Other

The project currently demonstrates specialized classes for:

- MedicalEmergency
 - FireEmergency
 - SecurityEmergency
-

3.4 Emergency Priority Calculation

The system should calculate the priority of an emergency according to its severity and type.

Severity levels include:

- LOW
- MEDIUM
- HIGH
- CRITICAL

The priority service assigns a numerical priority level to an emergency. Medical emergencies receive additional priority adjustment in the implemented logic.

3.5 Responder Assignment

The system should identify an available responder suitable for the type of emergency.

Examples include:

- Medical emergency → Medical Responder
- Fire emergency → Fire Responder
- Security emergency → Security Responder

Once assigned, the responder's availability is updated.

3.6 Resource Allocation

The system should manage emergency resources and allocate an available resource when required.

Resources used in the project include:

- Campus Ambulance
- First Aid Kit
- Fire Extinguisher

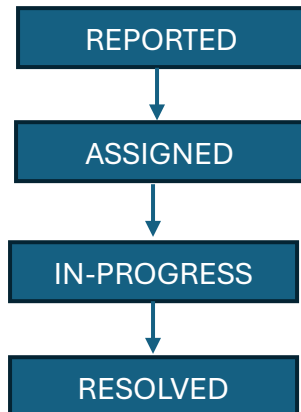
- Campus Security Vehicle

If a required resource is unavailable, the system generates a `ResourceUnavailableException`.

3.7 Emergency Tracking

The system should track the status of an emergency.

The supported status are:



The system can also identify an emergency using its Emergency ID and update its status.

3.8 Exception Handling

The system should handle expected errors without terminating the entire application.

The project implements:

- `EmergencyNotFoundException`
- `ResourceUnavailableException`

For example- if an invalid Emergency ID such as 999 is entered into the tracking operation an appropriate exception message is displayed.

3.9 Multithreaded Emergency Processing

The system should support processing of multiple emergencies using separate threads.

`EmergencyProcessingThread` is used to process emergency operations concurrently, including resource allocation, processing, resource release and status updates.

3.10 Emergency Report Generation

The system should save emergency information into a text file.

The generated report contains:

- Emergency ID
- Emergency type
- Severity
- Status

The system can also read and display the saved reports.

4. Non-Functional Requirements

4.1 Performance

The system should process emergency operations efficiently. Multiple emergency-processing operations can be executed using separate threads.

4.2 Reliability

The system should handle expected errors through exception handling and provide meaningful messages instead of abruptly terminating the program.

4.3 Usability

The console interface should clearly display:

- Emergency information
- Priority
- Resource status
- Responder assignment
- Processing status
- Error messages

4.4 Maintainability

The project is divided into separate packages and classes according to their responsibilities.

The major packages are:

model

enums

service

exception

interfacepackage

thread

This organization makes the project easier to understand and maintain.

4.5 Resource Management

The system should prevent an already allocated resource from being allocated again until it is released.

4.6 Error Handling

The system should provide meaningful error messages when an emergency cannot be found or when a required resource is unavailable.

5. System Architecture

CERA follows a **modular layered architecture**.

The system can be divided into the following major components:

5.1 Application Layer

Main.java

This is the entry point of the application. It coordinates the different services and demonstrates the complete emergency management workflow.

5.2 Model Layer

The model package contains the main entities of the system.

Examples:

- User
- Student
- Faculty
- Admin
- Responder
- MedicalResponder
- FireResponder
- SecurityResponder
- Emergency
- MedicalEmergency
- FireEmergency
- SecurityEmergency
- Resource

5.3 Enum Layer

The enums package contains predefined values for:

- EmergencyType
- Severity
- EmergencyStatus
- ResourceType

5.4 Service Layer

The service package contains the system's business logic.

Important services include:

- UserManagementService
- EmergencyPriorityService
- EmergencyAssignmentService
- ResourceManager
- EmergencyTrackingService
- EmergencyReportService

5.5 Exception Layer

The exception package contains custom exceptions:

- EmergencyNotFoundException
- ResourceUnavailableException

5.6 Interface Layer

The interfacepackage contains:

- Alertable

The interface is implemented by MedicalResponder to provide alert functionality.

5.7 Thread Processing Layer

The thread package contains:

- EmergencyProcessingThread

This class handles concurrent emergency processing.

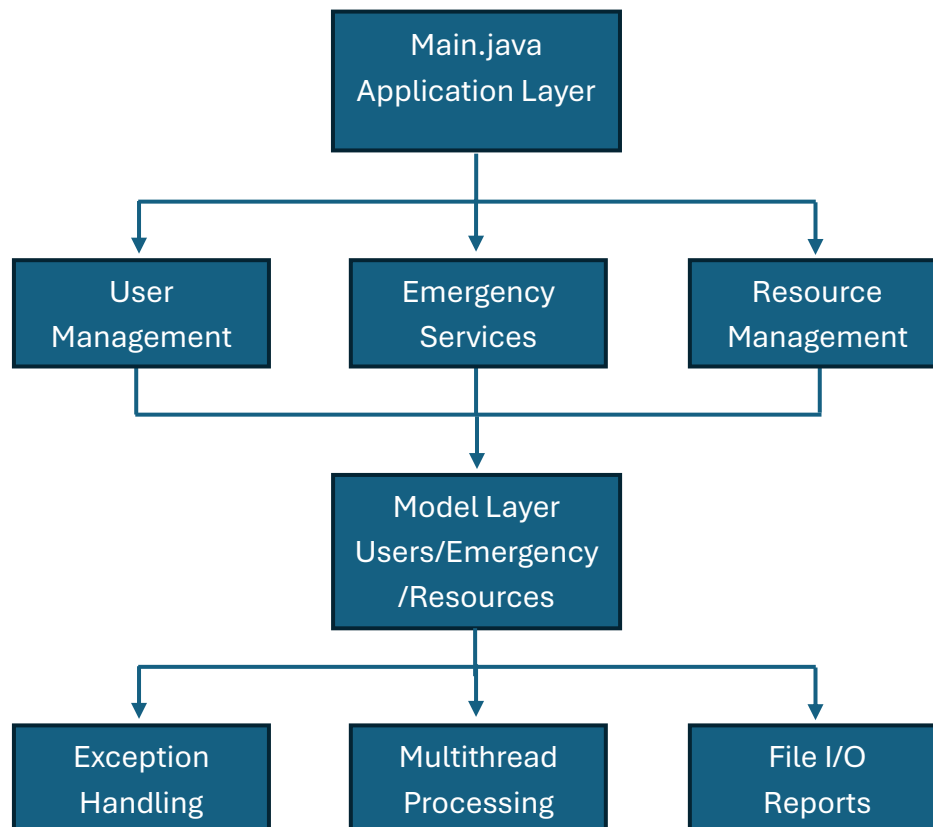
5.8 File Storage

The system uses:

emergency_report.txt

for storing generated emergency reports.

Architecture Diagram



7. Design Diagrams

7.1 Use Case Diagram

Actors

There are three primary types of users in the CERA design:

- Student/Faculty
- Responder
- Admin

Main Use Cases

Student/Faculty:

- Report Emergency
- View Emergency Status

Responder:

- View Assigned Emergency
- Respond to Emergency

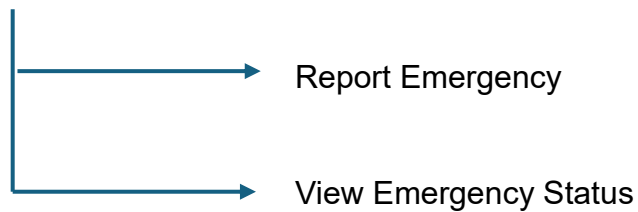
- Update Emergency Status

Admin:

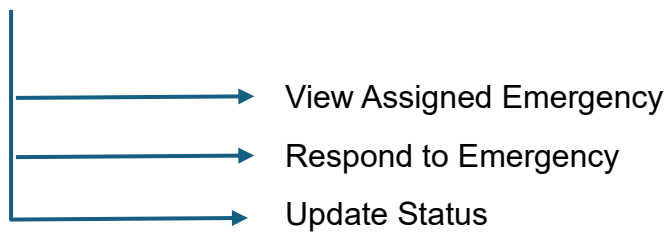
- Manage Users
- Manage Responders
- Manage Resources
- Monitor Emergencies
- View Reports

Use Case Diagram

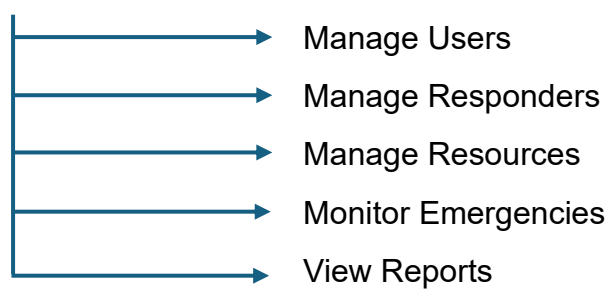
Student/Faculty



Responder



Admin



7.2 Workflow Diagram

The overall CERA workflow is:



7.3 Sequence Diagram

The sequence diagram represents interaction between the main components while processing an emergency.

Participants

- User
- Main
- EmergencyPriorityService
- EmergencyAssignmentService
- ResourceManager
- EmergencyProcessingThread
- EmergencyReportService

Sequence

User

|

| Report Emergency

↓

Main

|

| Create Emergency

↓

Emergency

|

| Send emergency details

↓

EmergencyPriorityService

|

| Calculate Priority

↓

Main

|

| Request responder

↓

EmergencyAssignmentService

|

| Find available responder

↓

Responder

|

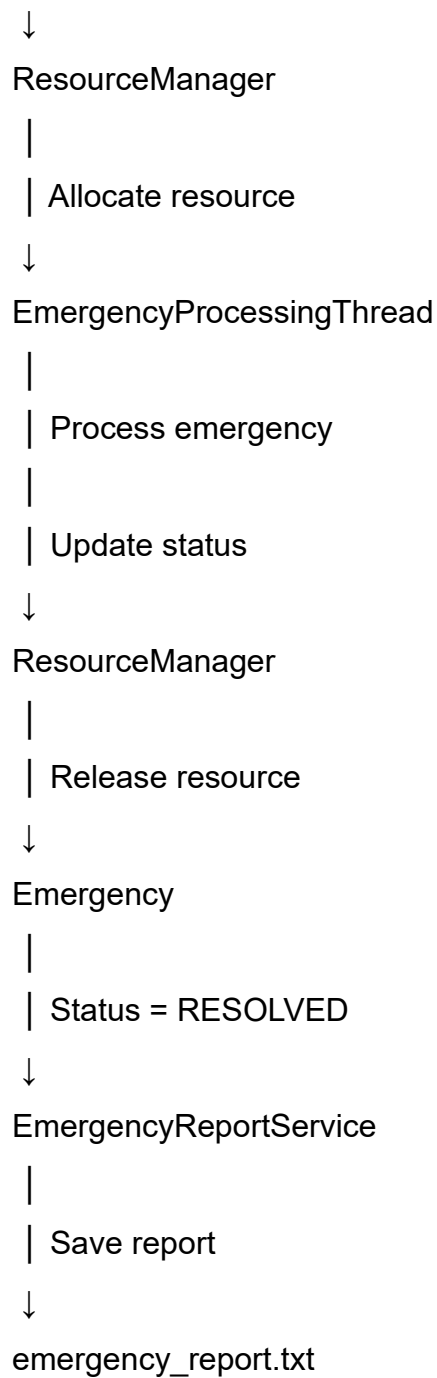
| Confirm response

↓

Main

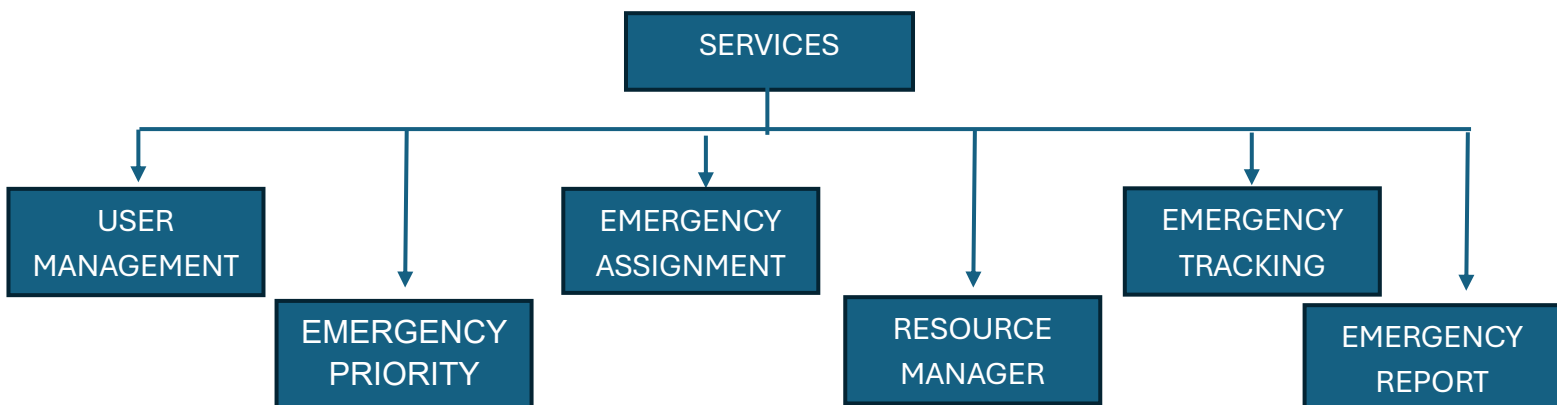
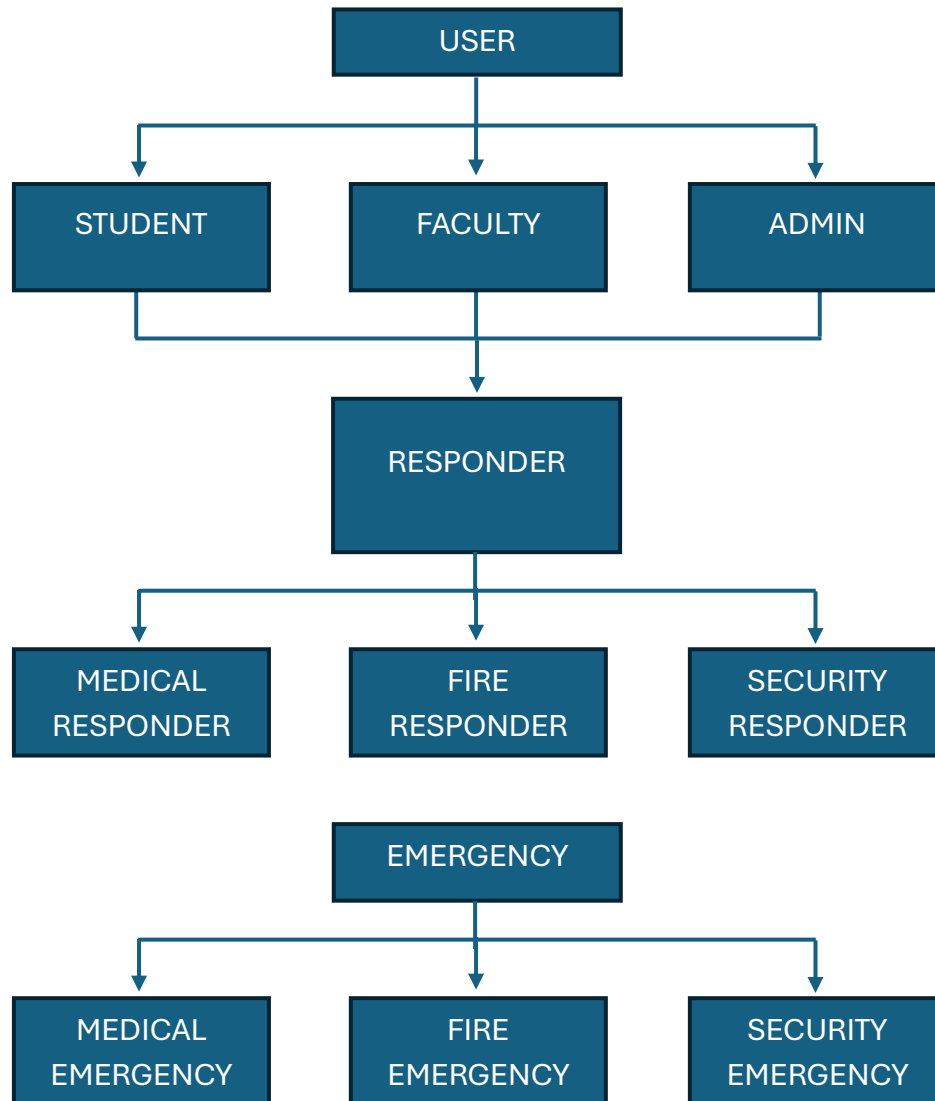
|

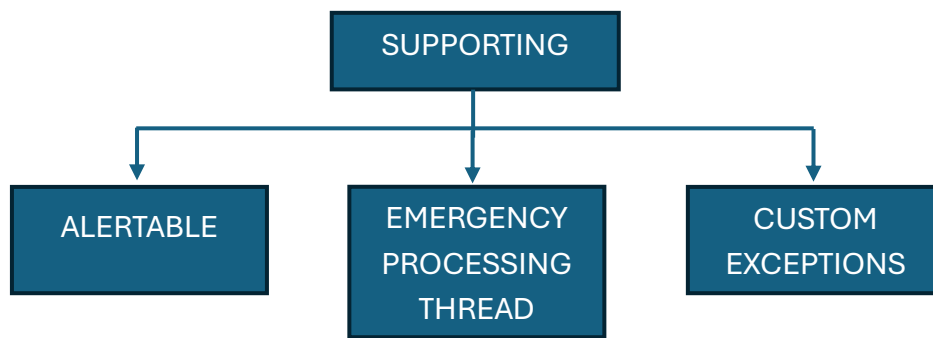
| Request resource



7.4 Class / Component Diagram

The main class relationships are:





8. Design Decisions & Rationale

8.1 Object-Oriented Design

The system is divided into classes representing real-world entities such as users, emergencies, responders, and resources.

This makes the system modular and easier to extend.

8.2 Abstraction

Emergency is implemented as an abstract class.

```
public abstract class Emergency
```

It contains common emergency information while allowing specialized emergency classes to define their own handling behaviour.

8.3 Inheritance

Inheritance is used to represent different types of users, responders, and emergencies.

For example:

User

├── Student

├── Faculty

├── Admin

└── Responder

 ├── MedicalResponder

 ├── FireResponder

 └── SecurityResponder

Similarly:

Emergency

└─ MedicalEmergency

└─ FireEmergency

└─ SecurityEmergency

8.4 Polymorphism

The `handleEmergency()` method is overridden by different emergency classes.

For example, a medical emergency performs medical-specific handling while a fire emergency performs fire-specific handling.

8.5 Interface

The `Alertable` interface defines an alert operation.

`MedicalResponder` implements the interface:

```
public class MedicalResponder extends Responder
    implements Alertable
```

This separates the alert behavior from the general responder functionality.

8.6 Enum Usage

Enums are used to represent fixed categories.

Examples:

`EmergencyType`

`Severity`

`EmergencyStatus`

`ResourceType`

This prevents inconsistent values and makes the program easier to understand.

8.7 Collections

Different Java collections are used according to their purpose.

- `ArrayList` → users and emergencies
- `Vector` → resources
- `Stack` → recent emergency tracking

8.8 Exception Handling

Custom exceptions are used for expected errors.

Examples:

EmergencyNotFoundException

ResourceUnavailableException

This allows the program to handle errors in a controlled manner.

8.9 Multithreading

EmergencyProcessingThread extends the Java Thread class.

Multiple emergency processing operations can therefore execute concurrently.

8.10 File Handling

File I/O is used to maintain emergency reports without requiring a database.

The file:

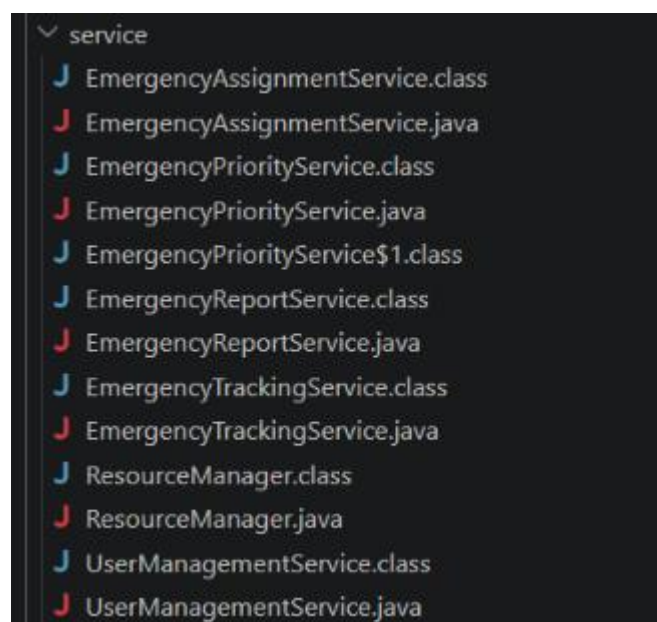
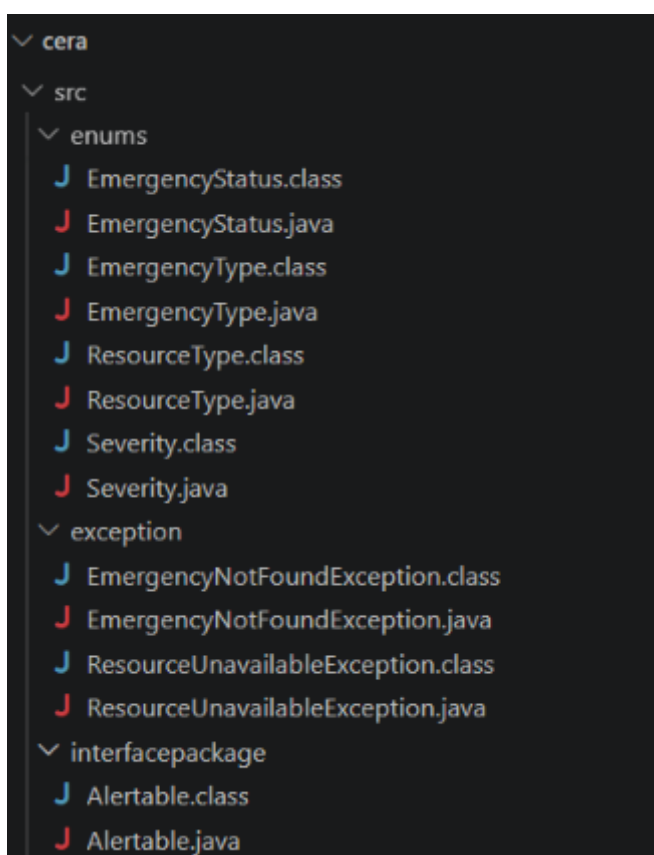
emergency_report.txt

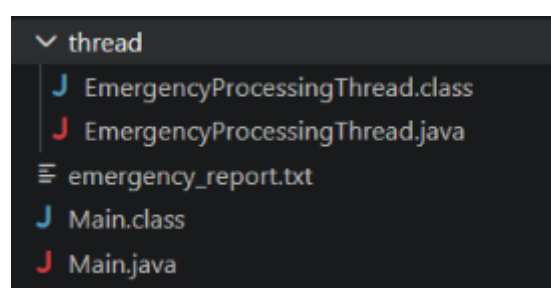
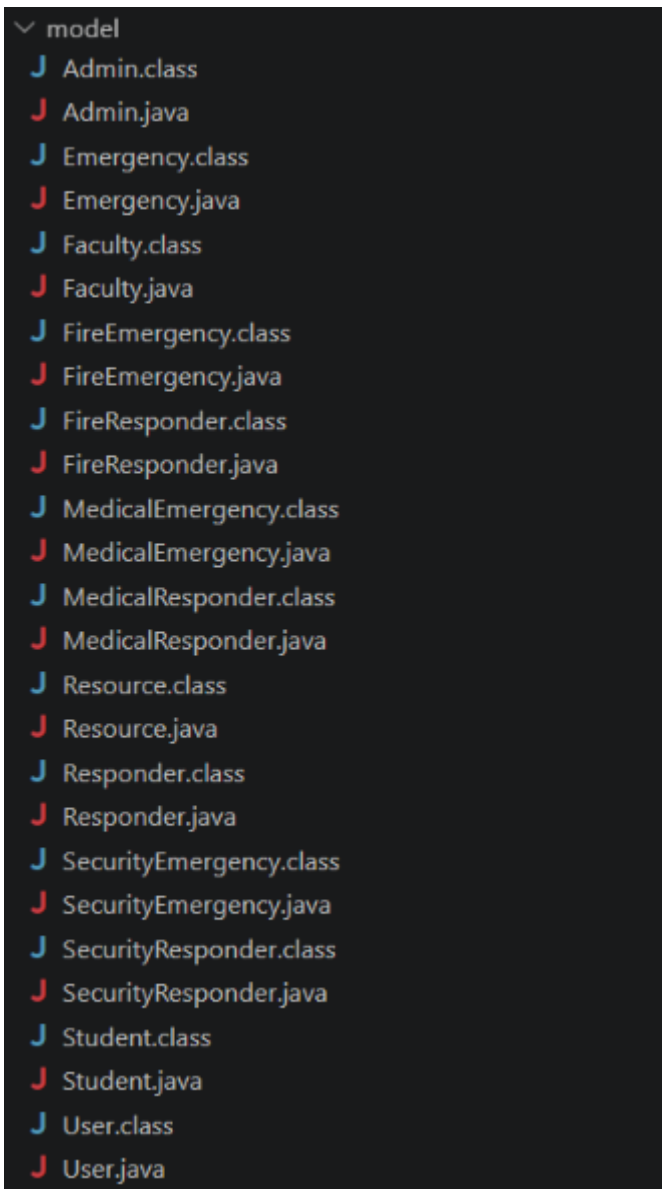
stores emergency information generated during program execution.

9. Implementation Details

CERA is implemented in Java using a modular package structure.

Project Structure





Major Implementation Components

Component	Purpose
Main.java	Controls overall program execution
Emergency	Abstract base emergency class
MedicalEmergency	Handles medical emergencies
FireEmergency	Handles fire emergencies
SecurityEmergency	Handles security emergencies
Responder	Base responder class
Resource	Represents emergency resources
EmergencyPriorityService	Calculates priority
EmergencyAssignmentService	Assigns responders

Component	Purpose
ResourceManager	Allocates/releases resources
EmergencyTrackingService	Tracks emergency status
EmergencyReportService	Saves/reads reports
EmergencyProcessingThread	Processes emergencies concurrently

10. Screenshots / Results

The CERA system was successfully executed and produced outputs for the major modules.

```

----- EMERGENCY DETAILS -----
Emergency ID : 101
Type       : MEDICAL
Location   : AB1 - Room 204
Severity   : HIGH
Description : Student injured
Status     : REPORTED
-----

----- EMERGENCY DETAILS -----
Emergency ID : 102
Type       : FIRE
Location   : Library - Ground Floor
Severity   : CRITICAL
Description : Smoke detected
Status     : REPORTED
-----

----- EMERGENCY DETAILS -----
Emergency ID : 103
Type       : SECURITY
Location   : Main Gate
Severity   : HIGH
Description : Unauthorized entry detected
Status     : REPORTED
-----

```

```

----- RESPONSE -----
Medical team dispatched for: Ambulance
Fire response team dispatched.
Campus evacuation is required.
Security team dispatched. Security level: HIGH

----- MEDICAL RESPONDER -----

----- RESPONDER -----
User ID : 201
Name    : Dr. Ankit
Email   : ankit@vit.ac.in
Available : true

-----
Specialization : Emergency Medicine
-----

Dr. Ankit is responding with medical assistance.
MEDICAL ALERT: Medical emergency reported at AB1.

----- ALL RESOURCES -----
Resource ID : 301
Name        : Campus Ambulance
Type        : AMBULANCE
Available   : true
-----
Resource ID : 302
Name        : First Aid Kit
Type        : FIRST_AID_KIT
Available   : true
-----
Resource ID : 303
Name        : Fire Extinguisher
Type        : FIRE_EXTINGUISHER
Available   : true
-----

```

```

Resource ID : 304
Name       : Campus Security Vehicle
Type       : SECURITY_VEHICLE
Available  : true
-----

Resource allocated: Campus Ambulance

ERROR: No AMBULANCE is currently available.

----- ALL RESOURCES -----
Resource ID : 301
Name       : Campus Ambulance
Type       : AMBULANCE
Available  : false
-----
Resource ID : 302
Name       : First Aid Kit
Type       : FIRST_AID_KIT
Available  : true
-----
Resource ID : 303
Name       : Fire Extinguisher
Type       : FIRE_EXTINGUISHER
Available  : true
-----
Resource ID : 304
Name       : Campus Security Vehicle
Type       : SECURITY_VEHICLE
Available  : true
-----

----- EMERGENCY PRIORITY -----

Emergency ID : 101
Emergency Type : MEDICAL
Severity : HIGH
Priority Level : 1

Emergency ID : 102
Emergency Type : FIRE
Severity : CRITICAL
Priority Level : 1

Emergency ID : 103
Emergency Type : SECURITY
Severity : HIGH
Priority Level : 2

----- MEDICAL RESPONDER -----

----- RESPONDER -----
User ID : 201
Name     : Dr. Ankit
Email    : ankit@vit.ac.in
Available : true
-----
Specialization : Emergency Medicine
-----

----- RESPONDER -----
User ID : 202
Name     : Rohit
Email    : rohit@vit.ac.in
Available : true
-----

----- RESPONDER -----
User ID : 203
Name     : Arjun
Email    : arjun@vit.ac.in
Available : true
-----

Resource released: Campus Ambulance

----- USER MANAGEMENT -----
User 1 added successfully.
User 2 added successfully.
User 3 added successfully.
User 201 added successfully.
User 202 added successfully.
User 203 added successfully.

----- ALL USERS -----

-----STUDENT -----
User ID : 1
Name     : Tavishi
Email    : tavishi@vit.ac.in
Course   : CSE AI-ML
-----

-----FACULTY -----
User ID : 2
Name     : Dr. Sharma
Email    : sharma@vit.ac.in
Department : Computer Science
-----

----- ADMIN -----
User ID : 3
Name     : Campus Admin
Email    : admin@vit.ac.in
Department : Emergency Management
-----

PS C:\Users\Tavishi\OneDrive\Documents\java\cera> cd "c:\Us

```

Checking user type...
Role: Student

Checking user type...
Role: Faculty

Checking user type...
Role: Admin

Checking user type...
Role: Responder

----- RESPONDER ASSIGNMENT -----
Responder 201 added to assignment system.
Responder 202 added to assignment system.
Responder 203 added to assignment system.

Responder assigned successfully.
Emergency ID : 101
Responder : Dr. Ankit
Dr. Ankit is responding with medical assistance.

Responder assigned successfully.
Emergency ID : 102
Responder : Rohit
Rohit is responding to the fire emergency.

Responder assigned successfully.
Emergency ID : 103
Responder : Arjun
Arjun is responding to the security emergency.

----- ALL EMERGENCIES -----

----- EMERGENCY DETAILS -----
Emergency ID : 101
Type : MEDICAL
Location : AB1 - Room 204
Severity : HIGH
Description : Student injured
Status : RESOLVED

----- EMERGENCY DETAILS -----
Emergency ID : 102
Type : FIRE
Location : Library - Ground Floor
Severity : CRITICAL
Description : Smoke detected
Status : RESOLVED

----- EMERGENCY DETAILS -----
Emergency ID : 103
Type : SECURITY
Location : Main Gate
Severity : HIGH
Description : Unauthorized entry detected
Status : RESOLVED

----- FILE I/O -----
Emergency report saved successfully.
Emergency report saved successfully.
Emergency report saved successfully.

----- SAVED REPORTS -----

----- MULTITHREADING -----

Thread started for Emergency 101

Thread started for Emergency 102

Thread started for Emergency 103

Resource allocated: Campus Ambulance

Resource allocated: Fire Extinguisher
Emergency 101 is being processed.
Emergency 102 is being processed.

Resource allocated: Campus Security Vehicle
Emergency 103 is being processed.

Resource released: Fire Extinguisher

Resource released: Campus Ambulance

Resource released: Campus Security Vehicle
Emergency 101 resolved.
Emergency 103 resolved.
Emergency 102 resolved.

All emergency threads completed.

----- EMERGENCY TRACKING -----

Emergency 101 added successfully.
Emergency 102 added successfully.
Emergency 103 added successfully.

Latest Emergency: 103

Emergency 101 status updated to ASSIGNED
Emergency 101 status updated to IN_PROGRESS
Emergency 101 status updated to RESOLVED
ERROR: Emergency ID 999 was not found.

----- FILE I/O -----
Emergency report saved successfully.
Emergency report saved successfully.
Emergency report saved successfully.

----- SAVED REPORTS -----

----- EMERGENCY REPORT -----
Emergency ID : 101
Type : MEDICAL
Severity : HIGH
Status : RESOLVED

----- EMERGENCY REPORT -----
Emergency ID : 102
Type : FIRE
Severity : CRITICAL
Status : RESOLVED

----- EMERGENCY REPORT -----
Emergency ID : 103
Type : SECURITY
Severity : HIGH
Status : RESOLVED

----- CERA PROGRAM COMPLETED -----

11. Testing Approach

Testing was performed by executing the major functional components and checking whether the actual output matched the expected behavior.

Test Case	Test Input/Condition	Expected Result	Status
TC01	Create medical emergency	Emergency created successfully	Pass
TC02	Create fire emergency	Fire emergency created	Pass
TC03	Create security emergency	Security emergency created	Pass
TC04	Calculate emergency priority	Correct priority displayed	Pass
TC05	Allocate available ambulance	Ambulance allocated	Pass
TC06	Allocate same unavailable ambulance	ResourceUnavailableException	Pass
TC07	Release resource	Resource becomes available	Pass
TC08	Assign medical responder	Medical responder assigned	Pass
TC09	Assign fire responder	Fire responder assigned	Pass
TC10	Assign security responder	Security responder assigned	Pass
TC11	Update valid emergency status	Status updated	Pass
TC12	Search Emergency ID 999	EmergencyNotFoundException	Pass
TC13	Start multiple processing threads	Emergencies processed concurrently	Pass
TC14	Save emergency report	Report saved to text file	Pass
TC15	Read saved reports	Reports displayed	Pass

Testing Result

The implemented test cases produced the expected results during program execution. Exception handling, resource allocation, responder assignment, emergency tracking, multithreading, and file-based report generation were successfully demonstrated.

12. Challenges Faced

12.1 Package Organization

Organizing classes into multiple packages required proper package declarations and imports.

12.2 Class Relationships

Creating relationships between users, responders, emergencies, and specialized subclasses required careful use of inheritance and polymorphism.

12.3 Resource Availability

The system needed to ensure that an allocated resource could not be allocated again until it was released.

12.4 Exception Handling

Custom exceptions had to be created and handled for situations such as unavailable resources and invalid emergency IDs.

12.5 Multithreading

Processing multiple emergencies through separate threads required coordination between emergency processing and resource management.

12.6 File Handling

The project required correct writing and reading of multiple emergency reports from a text file.

13. Learnings & Key Takeaways

The development of CERA provided practical understanding of several Java programming concepts.

Major Learnings

- Understanding object-oriented system design.
- Implementing classes and objects for real-world entities.
- Using **inheritance** to create specialized users, responders, and emergencies.
- Using **abstract classes** to define common emergency behavior.
- Implementing **method overriding and polymorphism**.
- Creating and implementing **interfaces**.
- Using **enums** for fixed categories.
- Using Java collections such as ArrayList, Vector, and Stack.
- Creating and handling **custom exceptions**.
- Implementing **multithreading** using the Thread class.

- Performing **file input/output** using FileWriter and FileReader.
- Organizing a project using Java packages.
- Designing a modular application instead of putting all functionality into one class.
- Testing individual system operations and handling expected errors.

Overall, the project provided practical experience in applying Java programming concepts to a real-world software problem.

14. Future Enhancements

The current version of CERA is a console-based Java application. The following features can be considered for future development:

14.1 Database Integration

A relational database could be introduced to provide persistent storage for users, emergencies, resources, responders, and reports.

14.2 Web-Based Interface

The console application could be converted into a web-based campus emergency management platform.

14.3 Mobile Application

A mobile application could allow students and faculty to report emergencies directly from their phones.

14.4 GPS Integration

Location services could automatically provide the location of an emergency.

14.5 Real-Time Notifications

SMS, email, or application notifications could be integrated to alert responders and administrators.

14.6 CCTV Integration

Campus CCTV systems could potentially be integrated for additional emergency monitoring.

14.7 Advanced Analytics

Historical emergency data could be analyzed to identify patterns and improve campus emergency planning.

15. References

Use only the references that you actually used while developing the project. A suitable format is:

1. **VITyarthi Java Project Guidelines**, provided as the project assignment guideline.
2. **Oracle Java Documentation** — Java language and API documentation.
3. Java documentation for **Object-Oriented Programming, Collections, Exception Handling, Multithreading and File I/O**.
4. Course notes and lecture material provided for the Java Programming subject.